CodeStreak

AI-Enabled Competitive Programming Practice Platform

MS-1: Problem Statement and Design Document

Roll Number	Team Member	Primary Responsibility
CS26MT015	Avishkar	Backend development, REST APIs, database
CS26MT017	Shivam	Frontend development and user experience
CS26MT023	Kiran	AI integration, testing, and CI support

Repository: https://github.com/cs26mt015/CODESTREAK

Document status: Proposed design. Implementation details and provider configuration will be confirmed during development.

Contents

1. Project Overview
2. Dataset Plan
3. User Roles and Sitemap
4. Requirements and Scope
5. System Architecture and Technology Stack
6. Functional Design
7. AI Hint Design and Evaluation
8. Database Design
9. API Design and Access Control
10. Security, Privacy, and Reliability
11. Testing and Acceptance Criteria
12. Milestones and Team Work Allocation
13. Deployment and Deliverables
14. Risks, Limitations, and Future Work
15. Conclusion

1. Project Overview

1.1 Website Name

CodeStreak — AI-Enabled Competitive Programming Practice Platform.

1.2 Project Description

CodeStreak is a web-based practice platform for university students and beginner-to-intermediate programmers. It offers a curated library of programming problems that can be filtered by topic and difficulty. Registered learners can record attempts, maintain problem status, set daily goals, track activity streaks, and review progress summaries. An AI-assisted feature provides progressive, problem-contextual hints and conceptual explanations. Administrators can create, edit, categorize, review, and publish problems.

1.3 Target Users

Role	Needs and permitted activities
Public visitor	Browse published problems and view public problem details without registration.
Learner	Register/login; record attempts and solved status; manage goals; view personal activity, streaks, and progress; request AI hints.
Administrator	Manage problem statements, metadata, topic tags, editorial content, and publication status through protected operations.

1.4 Goals and Scope

- Provide a searchable problem library with difficulty labels and topic tags.
- Support learner-reported attempts and solved status, daily goals, timezone-aware streaks, and progress summaries.
- Offer progressive AI hints through a protected backend integration.
- Provide administrator-managed content, role-based access control, automated tests, CI, and reproducible setup.

In scope: problem browsing, accounts, practice tracking, goals, streaks, progress, recommendations, administrator workflows, and AI hints.

Out of scope for the initial version: custom compiler, arbitrary code-execution sandbox, full hidden-test judging system, live contests, and complex social networking.

1.5 Key Assumptions

The first release records practice activity rather than executing or automatically judging submitted programs. Solved status is therefore learner-reported. The selected AI provider and exact model remain configuration decisions to be confirmed.

2. Dataset Plan

2.1 Source and Licensing

The initial dataset will consist of original problems authored and maintained by the project team. No external dataset URL or license is required for original team-authored content. If third-party content is added later, its source, license, attribution requirements, and modifications will be documented before use. Synthetic user and activity data may be generated for testing and demonstration and will be identified as such.

2.2 Target Dataset Composition

Difficulty	Target count	Share
Easy	20	40%
Medium	20	40%
Hard	10	20%

Total	50	100%

Topic coverage
Arrays
Strings
Searching and Sorting
Hashing
Two Pointers
Greedy Algorithms
Binary Search
Dynamic Programming

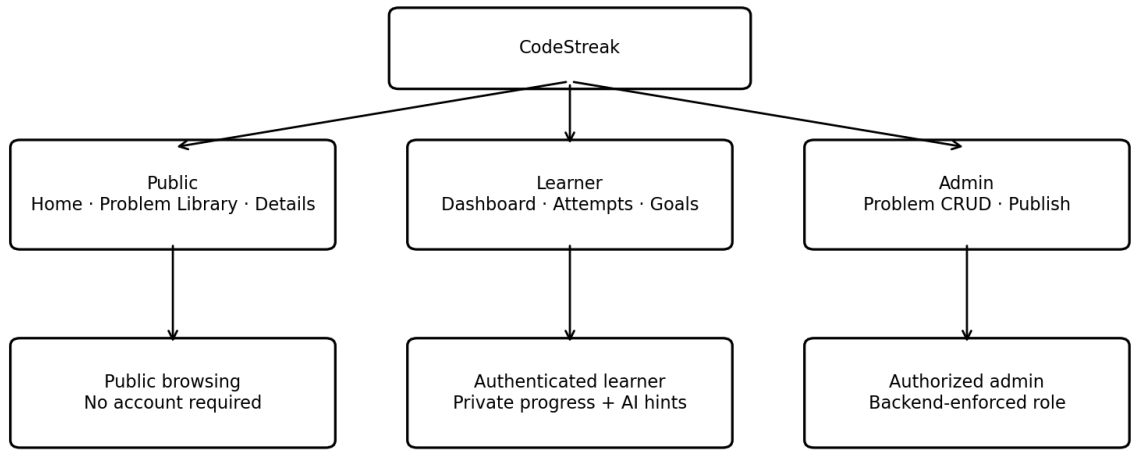
Each problem may have multiple topic tags. The seed dataset will be checked for topic and difficulty coverage, valid sample inputs/outputs, consistent constraints, and reviewed reference solutions.

2.3 Example Record

Field	**Illustrative value**
Problem ID / title	P001 — Sum of an Array
Difficulty / topics	Easy / Arrays
Purpose	Illustrative seed example; validate before inclusion
Editorial and reference solution	Stored for authorized internal/AI use; not exposed through public problem responses

3. User Roles and Sitemap

The application separates public browsing, authenticated learner workflows, and administrator-only content management. Backend authorization is authoritative; hiding a page in the frontend is not a security control.



Frontend visibility improves navigation; backend authorization protects every restricted operation.

Figure 1. Role-aware sitemap for public, learner, and administrator workflows.

3.1 Main Pages

Area	**Pages / functions**
Public	Home, problem library, filters, problem details, registration, login
Learner	Dashboard, problem practice, attempt/status update, daily goal, activity history, streaks, topic progress,

	recommendations, AI hints
Administrator	Problem list, create/edit problem, topic and difficulty management, preview/review, publish/unpublish

4. Requirements and Scope

4.1 Functional Requirements

- FR-1: Public users can browse and filter published problems by topic and difficulty.
- FR-2: Users can register and authenticate; private learner data is accessible only to its owner and authorized services.
- FR-3: Learners can record practice attempts and update their problem status.
- FR-4: The system calculates daily activity, current and longest streak, goal progress, and topic summaries using documented rules.
- FR-5: Learners can set a daily goal and maintain date-based practice plans.
- FR-6: The system recommends a suitable published problem that the learner has not solved, using deterministic rules.
- FR-7: Learners can request progressive AI hints subject to validation, configured limits, and provider availability.
- FR-8: Administrators can create, edit, review, and publish/unpublish problems.
- FR-9: The system provides API documentation, automated tests, and repeatable local setup.

4.2 Non-Functional Requirements

Area	Requirement
Security	Backend authentication and role enforcement; validate inputs; protect secrets; avoid exposing private editorial content.
Reliability	Controlled errors and graceful AI-provider timeout/rate-limit handling.
Maintainability	Modular frontend/backend, migrations, documented API contracts, and version-controlled configuration.
Usability	Responsive pages, clear validation/errors, and understandable progress summaries.
Testability	Unit/API/integration tests and browser-level tests for primary workflows.
Reproducibility	Documented environment variables, migrations, seed procedure, and Docker Compose setup.

5. System Architecture and Technology Stack

5.1 Logical Architecture

The system uses a client–server architecture. The frontend calls the backend REST API. The backend authenticates users, checks permissions, validates requests, applies business rules, reads/writes MySQL, and calls the external AI provider when required. The browser must not connect directly to the database or contain the AI provider secret.

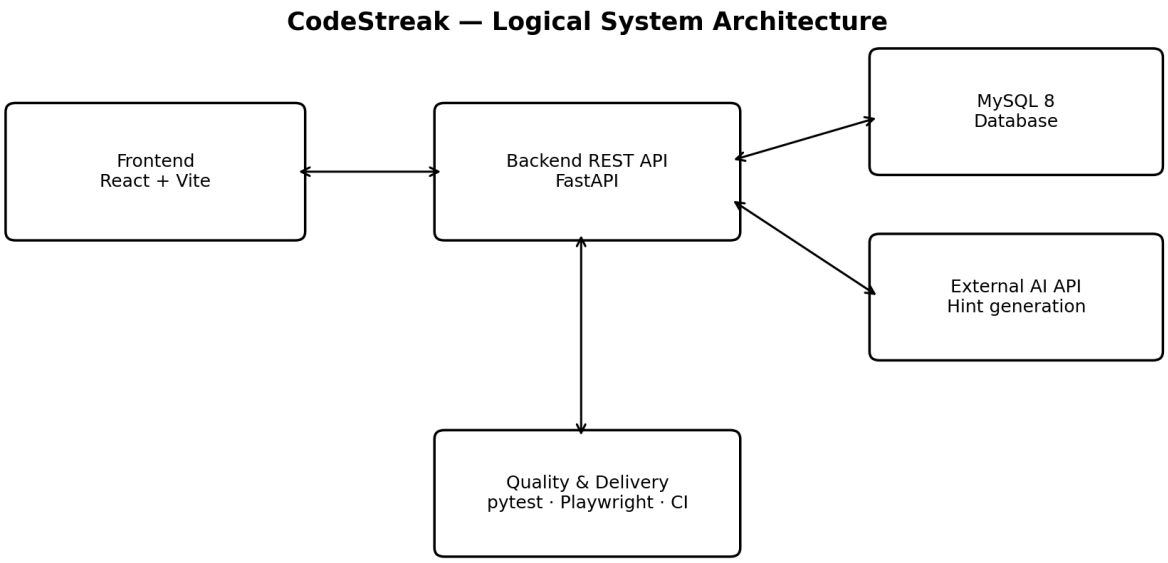


Figure 2. Logical system architecture and major service boundaries.

5.2 Technology Stack

Layer / purpose	Proposed technology
Frontend	React + Vite
Backend API	Python + FastAPI; REST; OpenAPI documentation
Database	MySQL 8
ORM / migrations	SQLAlchemy / Alembic
Backend testing	pytest and pytest-cov
End-to-end testing	Playwright
Version control / CI	Git, GitHub, GitHub Actions
Local environment	Docker and Docker Compose
AI hints	External AI API; provider/model to be confirmed

Exact runtime and dependency versions will be recorded in project configuration. Features described as proposed are not assumed to be implemented until verified in the repository.

6. Functional Design

6.1 Practice Attempts and Status

Learners record attempts against problem IDs and may update their problem status. Because there is no automatic judge in the initial release, the interface and reports must describe solved status as learner-reported rather than verified by execution.

6.2 Daily Activity, Goals, and Streaks

Concept	Rule
Qualifying active day	A local calendar day with at least one qualifying practice attempt, including an unsolved attempt.
Current streak	Count consecutive active local dates ending today if today is active; otherwise ending yesterday if yesterday is active; otherwise the current streak is 0.
Longest streak	Maximum consecutive sequence of active local dates in recorded activity.
Daily goal progress	Count distinct problems first marked solved on the local date; repeated attempts at the same problem do not increase this count.
Timezone	Use the learner's configured timezone when assigning local activity dates and evaluating date boundaries.

6.3 Recommendations

Recommendations will be deterministic: select from published, unsolved problems; prioritize topics and difficulty appropriate to the learner's practice context; and use a defined fallback when preferred candidates are unavailable. If no eligible problems remain, return a clear message rather than inventing a recommendation. The exact ranking weights can be set during implementation and documented.

6.4 Administrator Workflow

Authorized administrators can create and edit problem statements, constraints, samples, difficulty, tags, reference solutions, editorial notes, and publication status. Content should be reviewed before publication. All restricted operations must be checked in the backend.

7. AI Hint Design and Evaluation

7.1 Progressive Hint Levels

Level	Intended response
1 — Conceptual	Point toward the relevant concept or observation without prescribing the full algorithm.
2 — Approach	Suggest a suitable strategy and explain the main steps while leaving implementation decisions to the learner.
3 — Detailed guidance	Give more specific algorithmic guidance, invariants, edge cases, or pseudocode-like direction without automatically returning a complete reference solution.

7.2 Request Workflow

A learner requests a hint for a particular problem and level. The backend checks authentication, validates the problem and requested level, enforces rate/output limits, builds a minimal context from the public statement and approved internal editorial material, calls the configured AI provider, handles errors, and returns a controlled response. Learner source code will not be sent to the AI in the initial version.

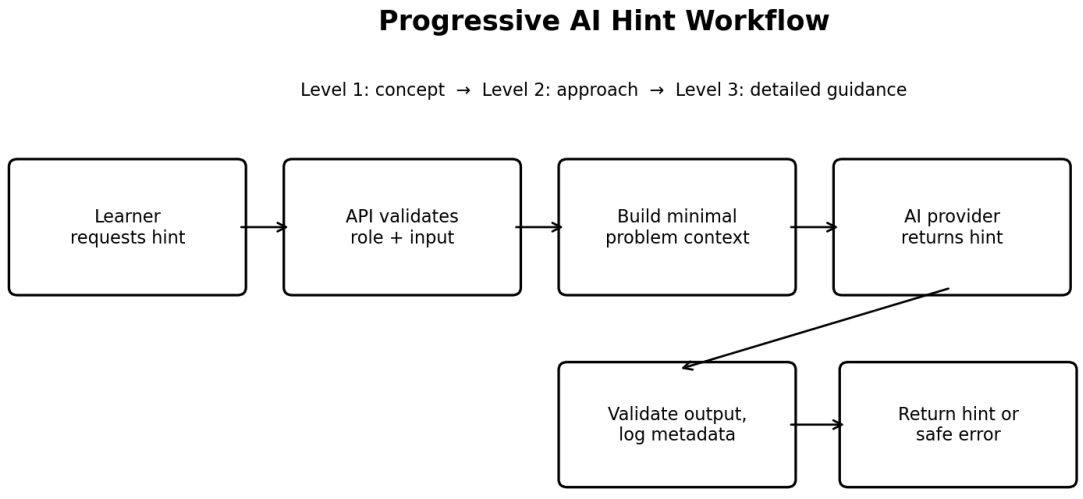


Figure 3. Progressive AI hint workflow with validation and failure handling.

7.3 Configuration and Safety Controls

- Keep provider credentials on the server and outside version control.
- Configure model/provider, request timeout, output-token/length limit, and per-user rate limits.

- Send only the minimum problem context needed; do not send passwords, tokens, or unnecessary account identifiers.
- Handle provider timeout, rate limit, malformed output, and temporary outage with safe, user-readable errors.
- Store only appropriate interaction metadata for evaluation and operations; avoid logging secrets or unnecessary private content.

7.4 Evaluation Plan

Evaluate at least 20 test requests using normal and adversarial prompts. Assess relevance to the problem, grounding in supplied context, progression between levels, algorithmic correctness, privacy, resistance to prompt injection, output-format compliance, and failure handling. Record test cases, expected behavior, observed responses, and unresolved issues. The selected provider/model is to be confirmed; no quality result is claimed before testing.

8. Database Design

8.1 Core Entities

The relational design separates user identity, problem content, topic mapping, learner status, attempt history, daily activity, practice goals/plans, and AI interaction metadata.

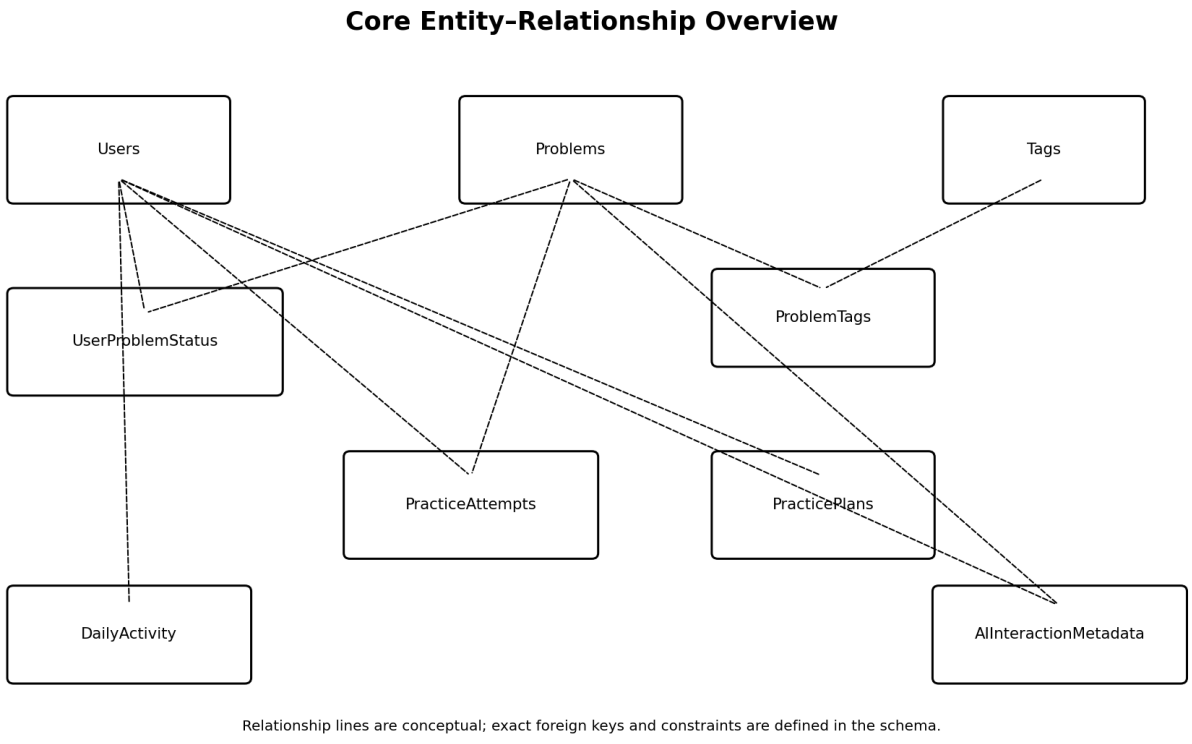


Figure 4. Conceptual ER overview; verify exact foreign keys and column types in the implementation schema.

8.2 Tables and Key Responsibilities

Table	Purpose / important fields
Users	Identity, authentication data, role, timezone, consent_status, timestamps.
Problems	Statement, constraints, samples, difficulty, editorial_notes, reference_solution, is_published, timestamps.
Tags	Canonical topic labels.
ProblemTags	Many-to-many mapping between problems and tags.
UserProblemStatus	Per-user status and solved-related timestamps for each problem.
PracticeAttempts	Attempt records, problem/user references, timestamp, and relevant attempt metadata.
DailyActivity	Per-user local-date activity summary used for streak/history calculations.
PracticePlans	Per-user daily goal/plan; enforce composite uniqueness on (user_id, plan_date).

AIInteractionMetadata	Minimal operational/evaluation metadata for AI hint requests.

8.3 Integrity and Exposure Rules

- Use primary keys, foreign keys, appropriate indexes, and constraints for referential integrity.
- PracticePlans must have a composite unique constraint on (user_id, plan_date), not a global unique constraint on plan_date.
- Public problem endpoints must not expose editorial_notes or reference_solution.
- Enforce ownership for learner-specific records and restrict problem management to administrators.
- Keep migrations and seed operations repeatable and safe against unintended duplicate inserts.

9. API Design and Access Control

9.1 API Conventions

The backend exposes REST endpoints and documents request/response schemas through OpenAPI. Endpoints should use consistent status codes, validation errors, pagination/filter conventions where applicable, and controlled error messages. Exact paths and schemas should be finalized with the implementation and kept synchronized with frontend API contracts.

9.2 Principal Endpoint Groups

Group	Representative operations	Access
Authentication	Register, login, current-user/session details, logout if session design requires it	Public for registration/login; authenticated for private account operations
Problems	List/filter published problems; retrieve public details	Public read access for published content
Practice	Create attempt; update per-problem status; view own attempt history	Authenticated learner; owner-scoped
Goals and activity	Set daily goal; view activity, streak, and progress summaries	Authenticated learner; owner-scoped
Recommendations	Retrieve next recommended published unsolved problem	Authenticated learner
AI hints	Request hint by problem and level	Authenticated learner; rate limited
Administration	Create/update/review/publish/unpublish problem content	Administrator only

9.3 Access-Control Matrix

Operation	Public	Learner	Admin
Browse published problems	Yes	Yes	Yes
View private personal progress	No	Own records	Only if explicitly required by policy
Create practice attempt / update own status	No	Own records	Not applicable as an admin privilege
Request AI hint	No	Yes	Only if allowed by product policy
Create/edit/publish problems	No	No	Yes
Read private editorial/reference solution through public API	No	No	Authorized internal/admin path only

The exact administrative visibility of learner data must follow the project's privacy requirements; administrator role must not imply unrestricted access to personal progress unless explicitly designed and justified.

10. Security, Privacy, and Reliability

- Authentication: use a suitable password/session or token approach and store credential material securely.
- Authorization: enforce roles and record ownership in backend dependencies/services for every restricted endpoint.
- Validation: validate all client inputs on the server, including IDs, levels, pagination, and content fields.

- Secrets: keep database credentials and AI API keys in environment configuration or a secret manager, never in frontend bundles or committed files.
- AI privacy: minimize transmitted context; initially do not send learner source code; avoid logging secrets and unnecessary personal data.
- Network and browser security: restrict CORS to intended frontend origins; use HTTPS for deployed traffic.
- Errors: return controlled messages without stack traces, database internals, or secret values.
- Availability: configure AI timeouts and rate-limit handling; keep core practice workflows useful when AI is unavailable.
- Persistence: use database volumes and consider backup/recovery where user data must survive container restarts.

11. Testing and Acceptance Criteria

11.1 Test Strategy

Test layer	Examples
Unit	Streak/date calculations, goal progress, recommendation ranking, validation and business rules.
API/integration	Authentication, access control, problem CRUD, attempt creation, owner isolation, database constraints, AI failure handling.
End-to-end	Public browsing; registration/login; record practice; view dashboard; admin creates and publishes a problem; request hint when configured.
AI evaluation	At least 20 normal/adversarial requests across hint levels; assess relevance, correctness, privacy, injection resistance, output compliance, and failure handling.
CI	Run configured backend tests and coverage; add frontend checks and browser tests where supported by the pipeline.

11.2 Acceptance Criteria

- Published problems are browseable without authentication; unpublished problems are not exposed publicly.
- Learners cannot access another learner's private records or administrator-only operations.
- Public problem responses omit private editorial notes and reference solutions.
- Streak and daily-goal calculations pass tests for consecutive/missed days, multiple attempts, and timezone boundaries.
- AI requests validate inputs, enforce configured limits, and handle provider failures safely.
- Primary learner and administrator workflows pass their end-to-end tests.
- Tests can be run using documented instructions, and known critical failures and limitations are recorded.

12. Milestones and Team Work Allocation

12.1 Milestones

Milestone	Main activities	Deliverable / exit condition
MS-1: Problem statement and design	Scope, requirements, dataset plan, sitemap, architecture, schema, API design	Reviewed design document
MS-2: Core backend	FastAPI foundation, models, migrations, authentication, problem APIs, access control	Tested backend foundation and documented endpoints
MS-3: Frontend and integration	Problem library/details, login/registration, frontend API integration	Working browsing and account workflows
MS-4: Practice tracking	Attempts/status, goals, streaks, activity, dashboard, recommendations	Working learner progress workflows
MS-5: AI hint integration	Provider configuration, progressive hints, validation, limits, error handling, evaluation	Hint workflow and evaluation evidence
MS-6: Stabilization	Integration tests, security checks, CI, bug fixing	Passing critical tests and documented issues
MS-7: Final delivery	README, demo preparation, final review and submission	Reproducible project and final documentation

Exact calendar dates and durations are intentionally not invented. The team will align this sequence with official course deadlines and adjust dependencies as needed.

12.2 Work Allocation

Team member	Primary responsibility
CS26MT015 — Avishkar	FastAPI endpoints, database schema/models, migrations, business logic, authentication, backend tests.
CS26MT017 — Shivam	React interface, layouts, navigation, forms, dashboard, progress visualization, frontend integration.
CS26MT023 — Kiran	AI provider integration, hint evaluation, test planning, end-to-end testing support, GitHub Actions.

Responsibilities are primary ownership areas rather than isolated work. Team members will coordinate through GitHub issues, branches, pull requests, and API/schema reviews.

12.3 Repository Practices

Repository: https://github.com/cs26mt015/CODESTREAK

- Use feature branches for new work and bug fixes; use pull requests and reviews where practical.
- Review API and database changes before integrating dependent frontend work.
- Keep real secrets out of the repository; provide an example environment file without secret values.
- Run relevant automated checks before merging and update documentation when contracts or setup change.

13. Deployment and Deliverables

13.1 Environment and Setup

The proposed development environment uses Docker Compose for repeatable services, with a documented alternative for local execution if provided. The README should describe prerequisites, environment variables, database startup, migrations, seed loading, backend/frontend startup, and test execution. Exact commands and ports must match the implemented configuration.

1. Clone the repository and configure environment variables using the example template.
2. Start MySQL and application services using the documented workflow.
3. Apply Alembic migrations and load the curated problem seed data.
4. Start the FastAPI backend and React frontend.
5. Verify core workflows and run the automated test suite.

13.2 Deliverables

Deliverable	Expected content
Source code	React frontend and FastAPI backend
Database	SQLAlchemy models, Alembic migrations, and initial problem seed
API documentation	OpenAPI specification generated from the backend
Testing evidence	Unit/API/integration and applicable end-to-end results; coverage report
Deployment configuration	Dockerfiles, Compose configuration, environment templates as implemented
Documentation	Requirements/design, architecture, schema, API and README setup/testing steps
Demonstration	Public browsing, learner practice/progress, admin content management, AI hints when configured
Known limitations	Incomplete features, provider dependencies, and unresolved issues

13.3 Definition of Completion

The project is ready for demonstration when a clean environment can follow the documented setup, initialize the database, run the core workflows, and execute the test suite. Any incomplete feature or external-service dependency will be disclosed rather than represented as complete.

14. Risks, Limitations, and Future Work

14.1 Risks and Mitigations

Risk / limitation	Mitigation or disclosure
No automatic judging	Clearly label status as learner-reported; defer compiler/sandbox and hidden-test judging.
AI service unavailable or costly	Use timeouts, rate limits, configurable output limits, graceful failure, and documented provider dependency.
AI response is inaccurate or reveals too much	Use progressive hints, minimal context, a 20-request evaluation set, and explicit quality/privacy checks.
Unauthorized access or data exposure	Backend role/ownership checks, validation, private-field filtering, and security tests.
Inconsistent streaks across timezones	Use user timezone consistently and test day boundaries and missed days.
Dataset quality or licensing issue	Use team-authored problems initially; review examples, constraints, solutions, and any later third-party license.
Scope exceeds course timeline	Prioritize core browsing, accounts, practice tracking, data integrity, and access control before refinements.

14.2 Future Enhancements

Possible future work includes a secure code-execution/judging system, richer adaptive recommendations, a larger reviewed problem library, follow-up AI tutoring, longer-term learning analytics, contest mode, external platform integration, and production-grade monitoring, backup/recovery, accessibility, and load testing. These are not commitments for the initial release.

14.3 Initial Release Priorities

Prioritize published problem browsing, authentication and role enforcement, learner-reported practice tracking, timezone-aware goals/streaks, deterministic recommendations, protected progressive AI hints, administrator content workflows, automated tests, and reproducible setup.

15. Conclusion

CodeStreak proposes a manageable AI-assisted environment for competitive programming practice. Its core combines a curated problem library, learner-reported practice tracking, goals, streaks, progress summaries, deterministic recommendations, and progressive AI hints. React and Vite provide the frontend, FastAPI exposes protected REST services, and MySQL stores application data. The design emphasizes clear scope, role-based access, private editorial content, timezone-consistent activity rules, controlled AI integration, testing, and reproducible setup. Automated judging and advanced tutoring remain future possibilities rather than initial-release promises.